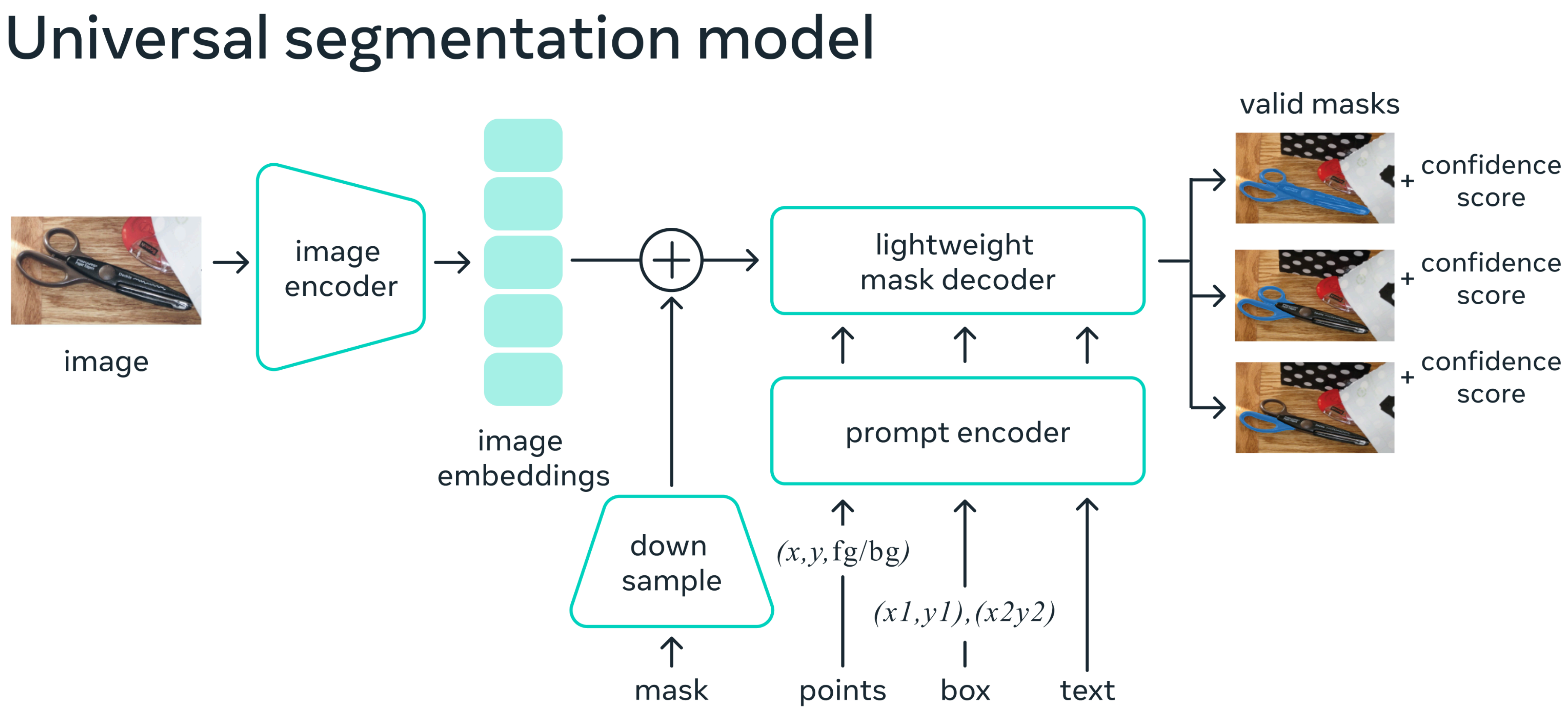
Deep Learning - Assignment 4

Segment Anything Fine-Tuning – Dental Dataset.

Guilherme Castilho, Pedro Tokar e Vitor do Nascimento

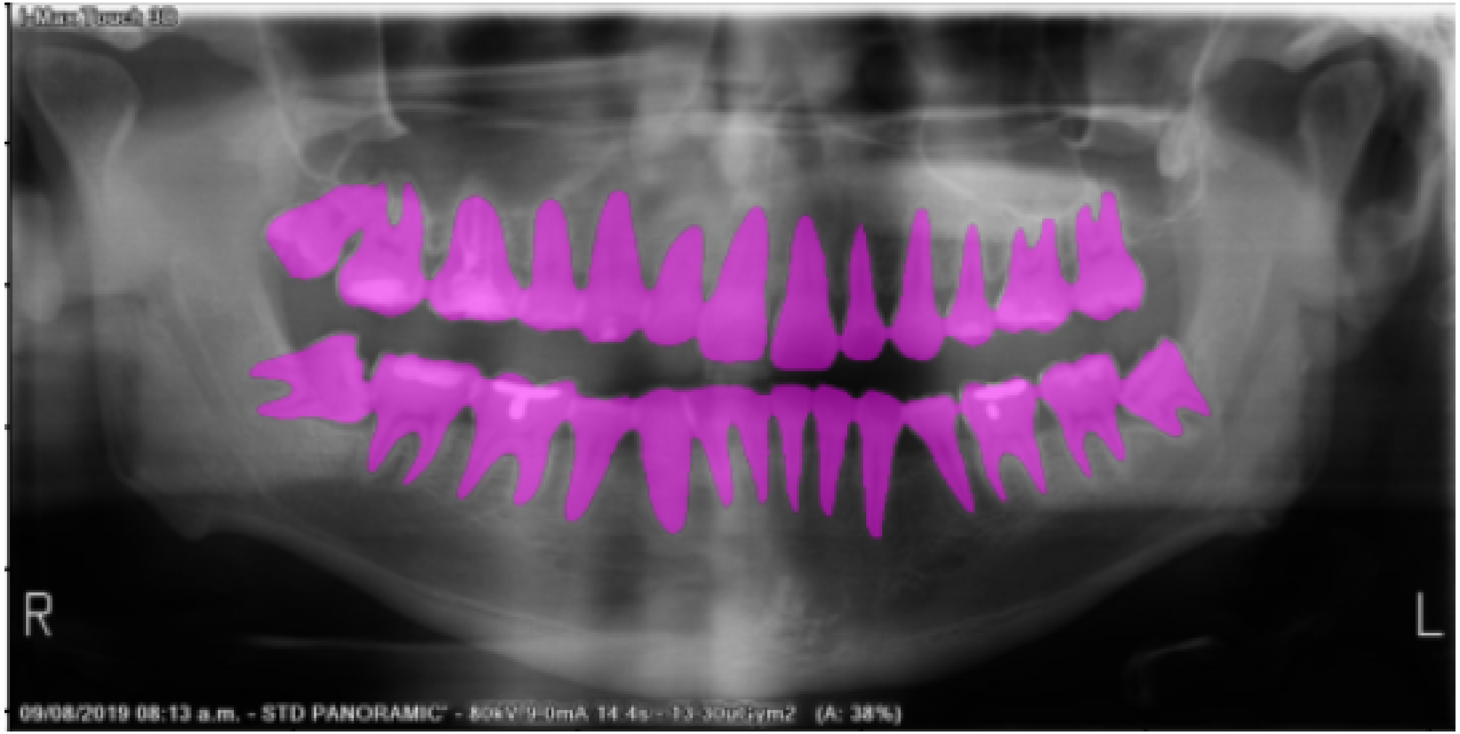
Sobre o SAM

Arquitetura

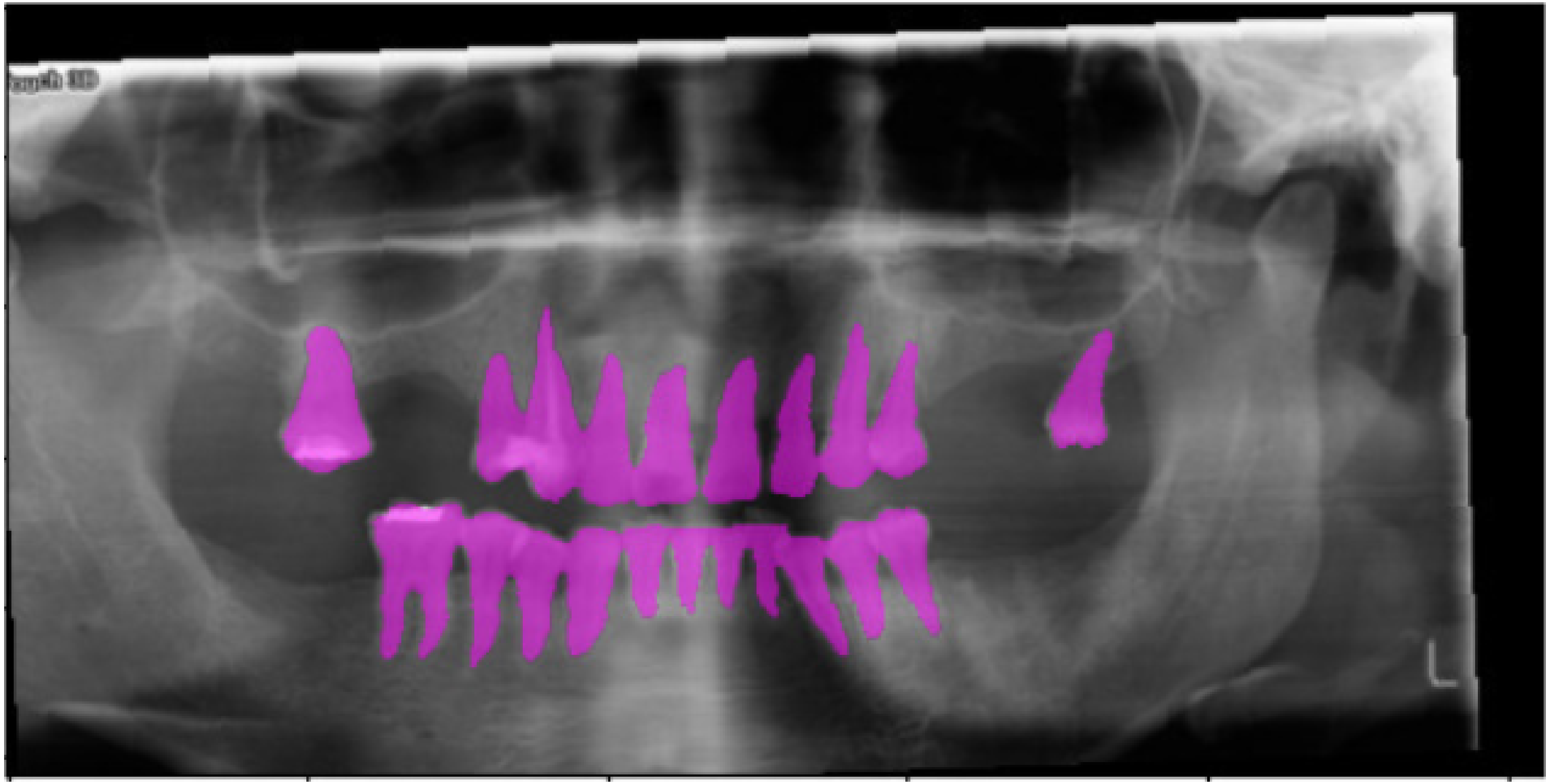


Escolha do problema

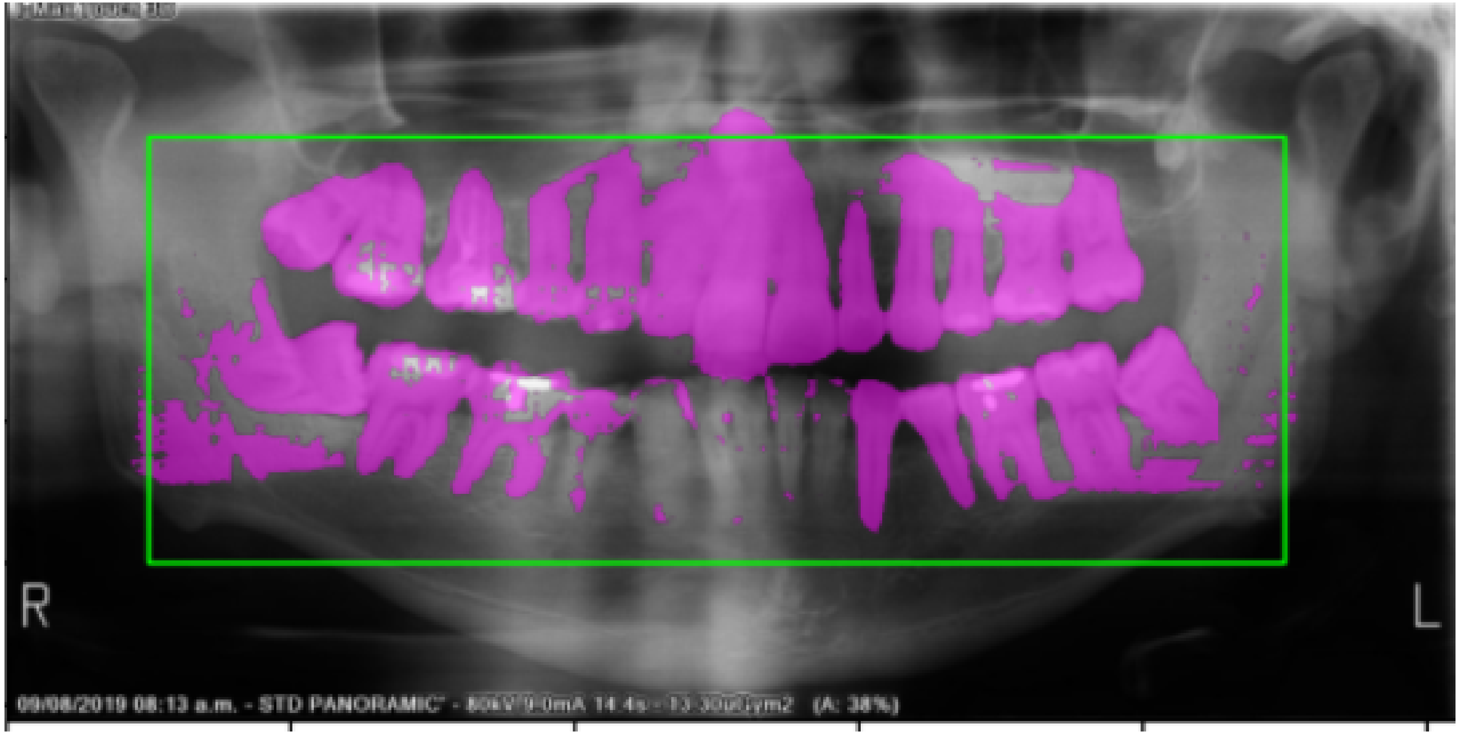
Dataset - exemplo de dado



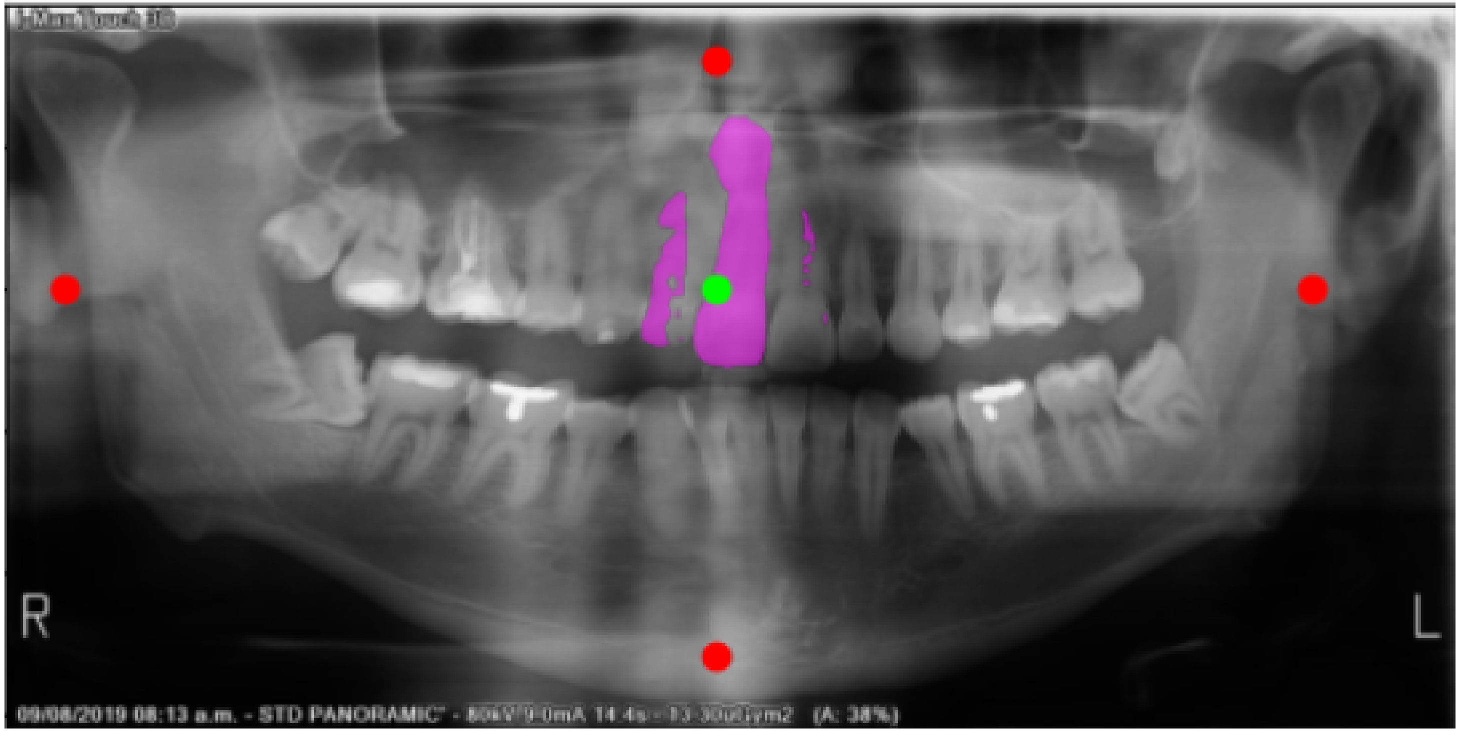
Dataset - exemplo augmentation



Resultado sem finetuning - box prompt



Resultado sem finetunig - point prompt



Modificações para o treinamento

Função de treino


```python
def train_one_epoch(self, train_loader, epoch):
    self._model.train()
    running_loss = 0.0

    for i, (images, gt_masks, boxes) in enumerate(progress_bar):
        self._optimizer.zero_grad()

        gt_masks = gt_masks.to(self._device)
        images = self._model.preprocess(images.to(self._device))

        with torch.no_grad():
            image_embeddings = self._model.image_encoder(images)

        loss = 0.0

        for j, (embedding, gt_mask, box) in enumerate(zip(image_embeddings, gt_masks, boxes)):

            with torch.no_grad():
                sparse_embeddings, dense_embeddings = self._model.prompt_encoder(
                    points = None,
                    masks = None,
                    boxes = torch.tensor(box).to(self._device).view(1, 1, 4)
                )
```

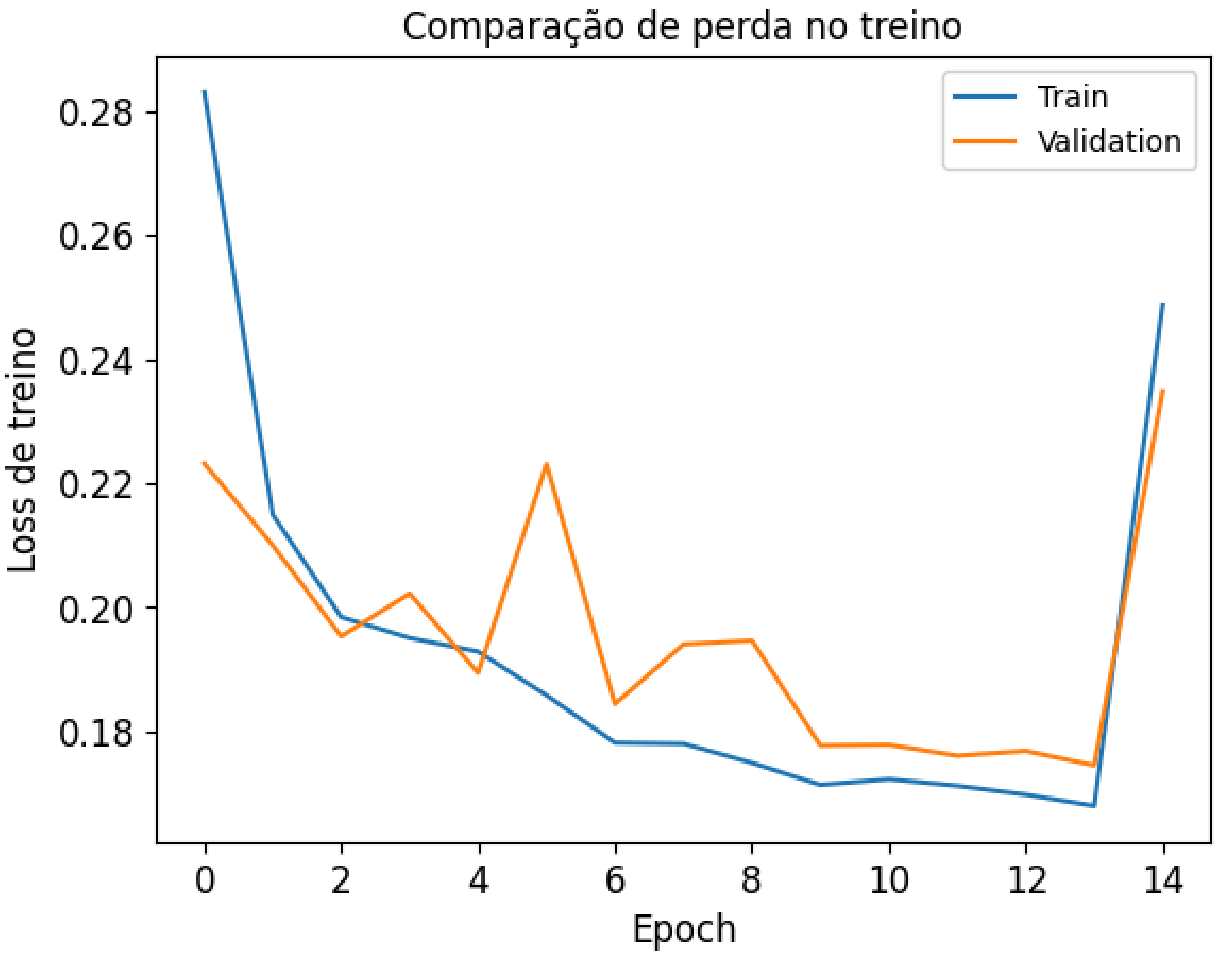
Função de treino


```
        low_res_masks, iou_predictions = self._model.mask_decoder(
            image_embeddings = embedding.unsqueeze(0), # (1, 256, 64, 64)
            image_pe = self._model.prompt_encoder.get_dense_pe(), #encodings posicionais
            sparse_prompt_embeddings = sparse_embeddings,
            dense_prompt_embeddings = dense_embeddings,
            multimask_output = False,
        )

        h_gt, w_gt = gt_mask.shape[-2], gt_mask.shape[-1]

        pred_mask = F.interpolate(
            low_res_masks[:, :, :128, :],
            size = (h_gt, w_gt),
            mode = "bilinear",
            align_corners = False
        ).squeeze(0)

        loss += self._criterion(pred_mask.float(), gt_mask.float())
        #[...]

    epoch_loss = running_loss / len(train_loader)
    return epoch_loss
```

Parâmetros do treinamento

- Learning rate: 10^{-3}
- Loss: BCE + Dice Loss
- Número de épocas: 15
- Otimizador: Adam com momento de Nesterov
- Batch size: 3

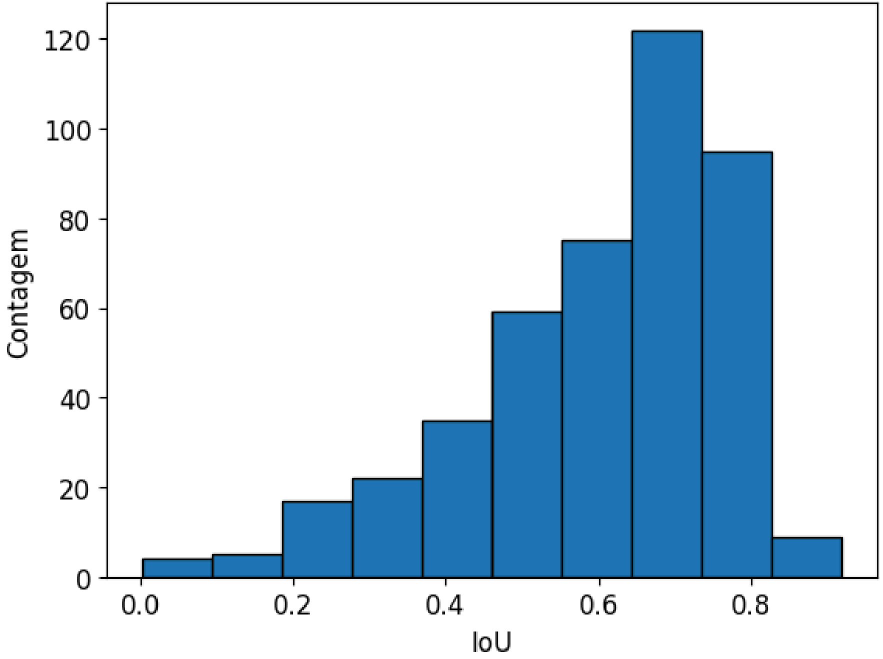
Histórico do fine tuning



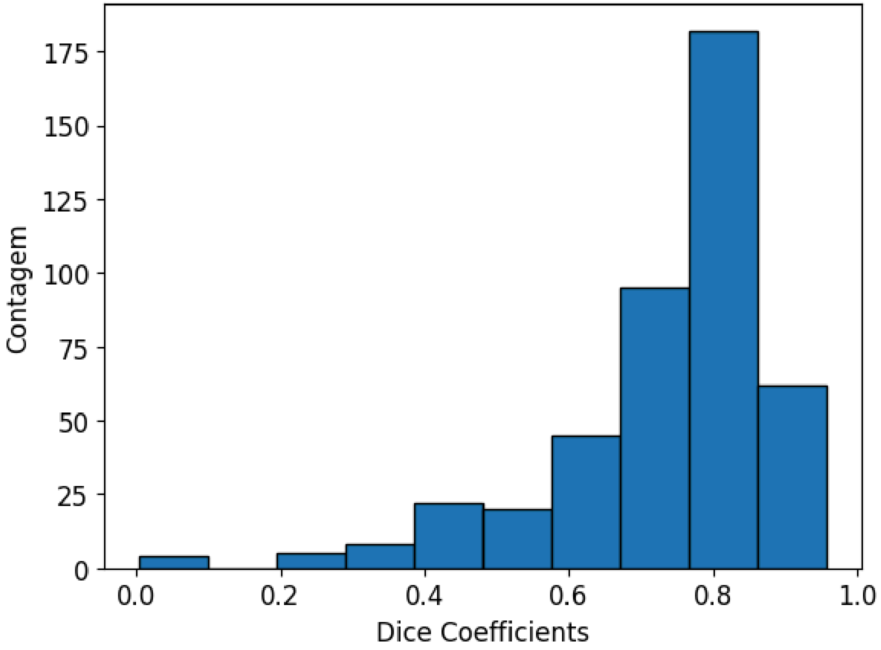
Resultados

IoUs e dice coefficients - modelo pré treinado

Média IoUs: 0.6034

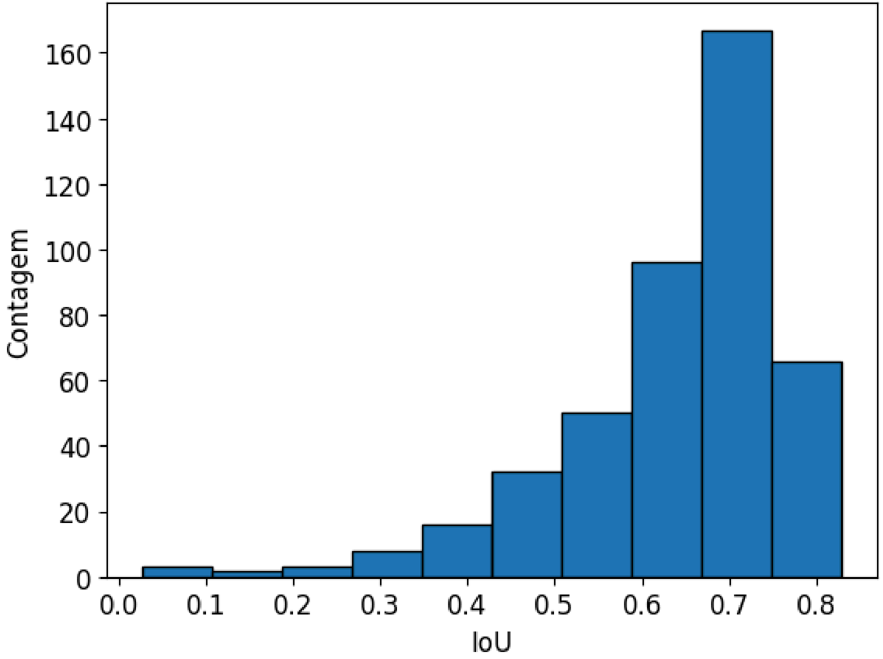


Média Dice coefficients: 0.7363

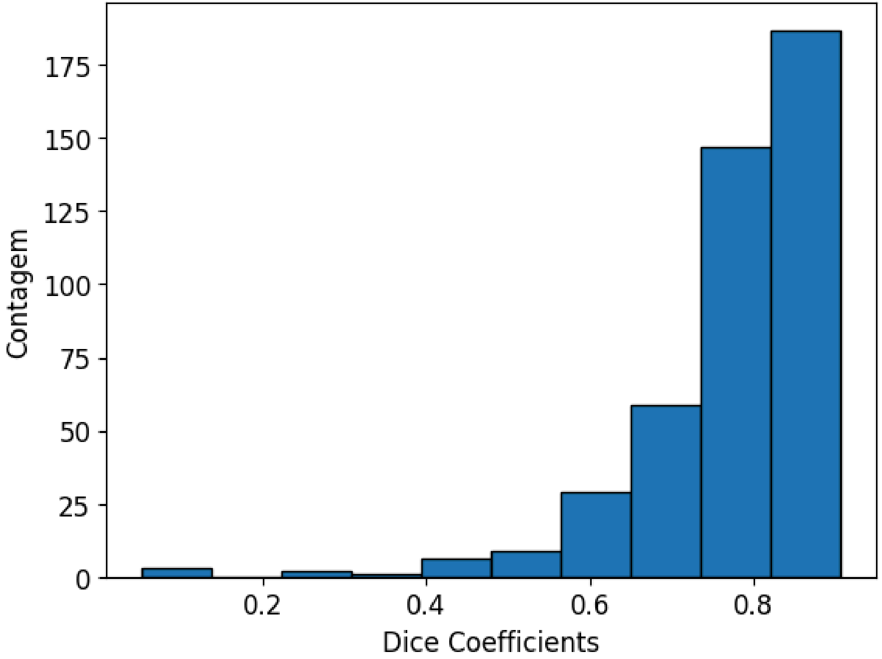


IoUs e dice coefficients - modelo com fine tuning

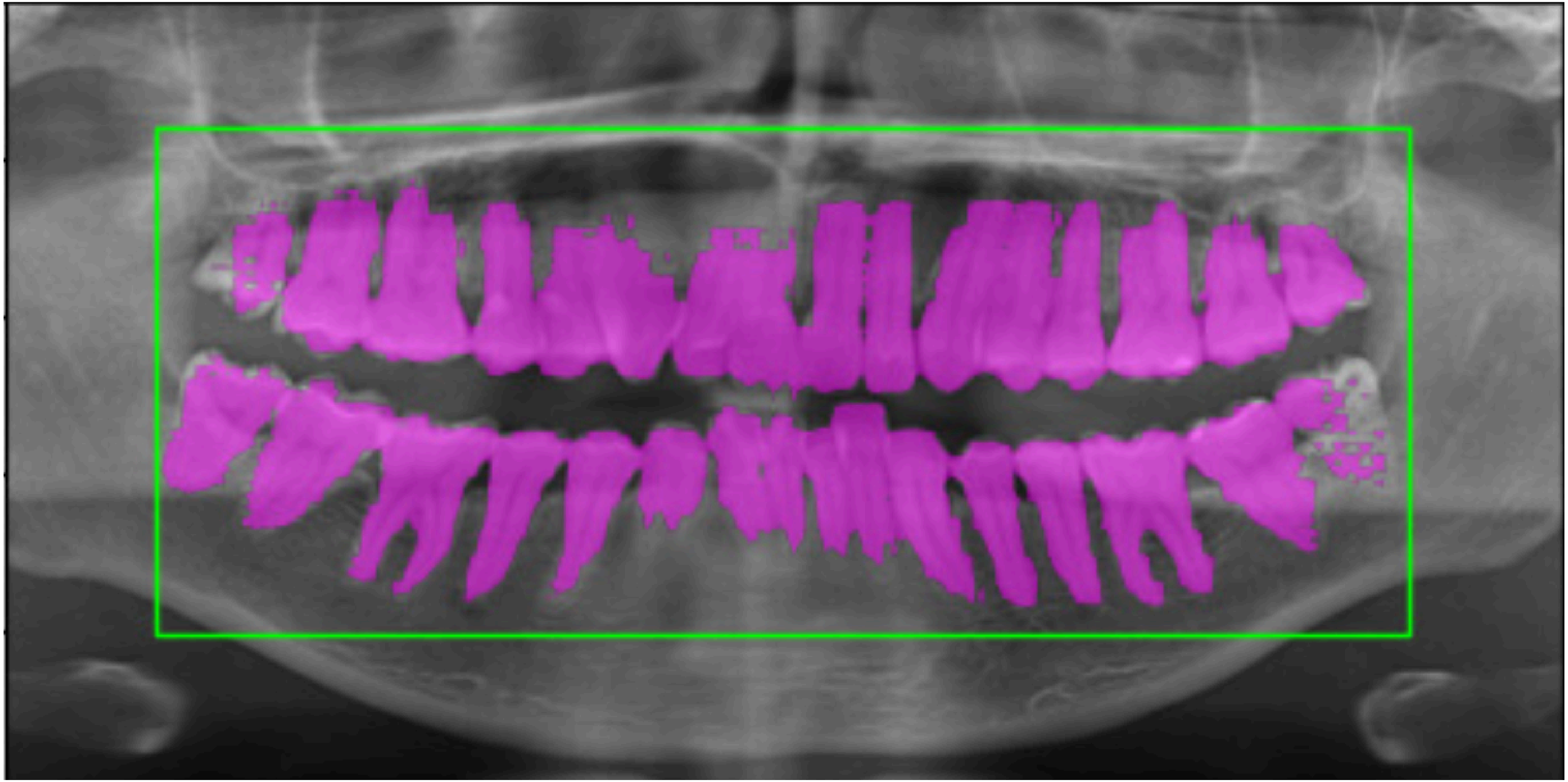
Média IoUs: 0.6404



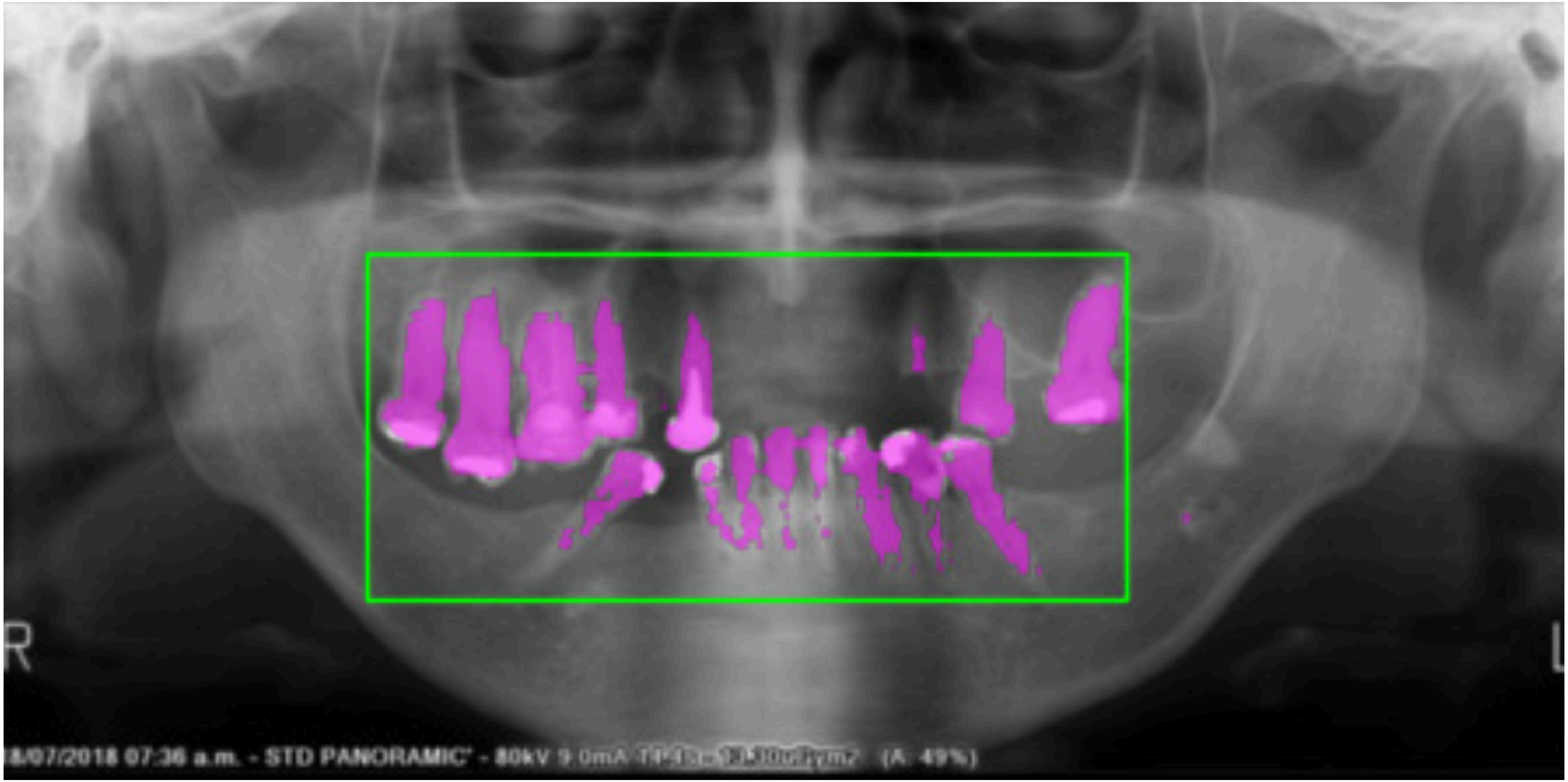
Média Dice coefficients: 0.7721



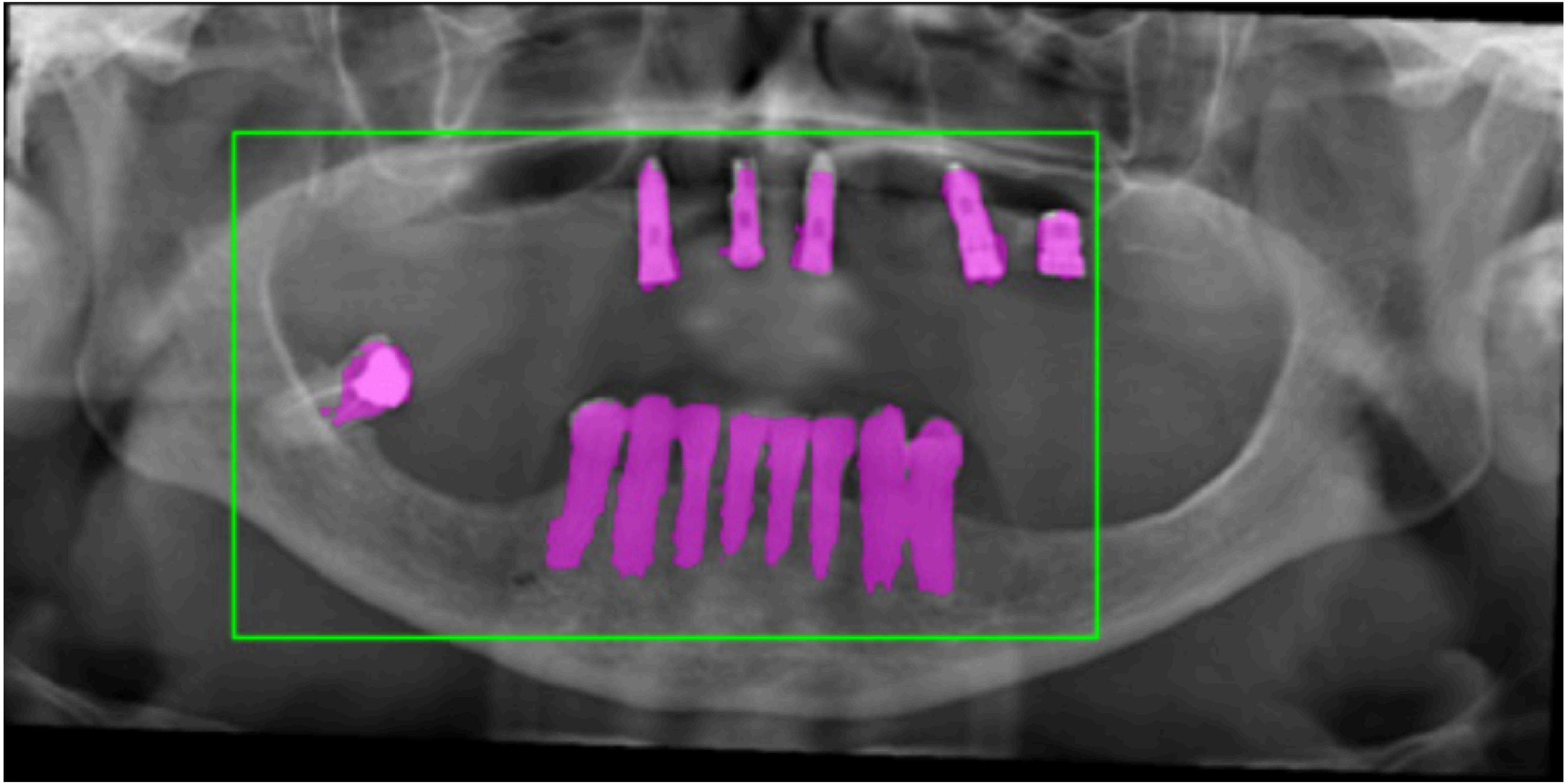
Exemplos de predições do modelo com fine tuning



Exemplos de predições do modelo com fine tuning



Exemplos de predições do modelo com fine tuning



Obrigado!